



CryptEX Whitepaper

A peer-to-peer monetary system with SHA3-512 proof-of-work and adaptive liveness control.

Implementation reference for the current CryptEX protocol, node, wallet, miner, and desktop client Version: April 3, 2026

Abstract

CryptEX is a UTXO-based cryptocurrency implemented primarily in C++ with a backend daemon, a separate Qt desktop client, a mining engine, and a JSON-RPC control surface. It combines SHA3-512 proof of work, full-node validation, a Boost.Asio-based peer-to-peer network, encrypted HD wallet storage, headers-first chain synchronization, binary on-disk blockchain persistence, and a conservative operator-security posture inspired by Bitcoin-style systems.

The strongest design claim of CryptEX is not merely that it is a new proof-of-work chain, but that it is engineered to remain operable under discontinuous hash-rate conditions. In smaller or still-improving proof-of-work networks, the dominant practical failure mode is often not cryptographic breakage but liveness collapse: a strong miner exits, difficulty remains too high, and the chain effectively stalls. CryptEX addresses that problem with a hybrid difficulty controller that combines weighted historical response, exponential smoothing, real-time overdue easing, and an emergency minimum-difficulty recovery rule. CryptEX treats liveness under discontinuous hash-rate as a first-class consensus concern, not a secondary tuning detail.

This whitepaper is not a speculative design memo. It describes how the current CryptEX implementation behaves today. Where the implementation is intentionally simplified, still improving, or still evolving, that is stated directly.

1. Design Goals

CryptEX is designed around the following goals:

1. Use SHA3-512 as the network's proof-of-work primitive.
2. Keep a Bitcoin-like full-node architecture with local validation, UTXO tracking, and cumulative-work chain selection.
- 3.

Provide a backend executable that can act as wallet, miner, node, and RPC server, while also supporting a separate desktop GUI client. 4. Keep network operation practical for small and still-improving deployments with built-in peer discovery, peer persistence, and direct miner interfaces. 5. Store blockchain and wallet data in system application-data locations by default, so deleting the binary does not delete the chain. 6. Preserve liveness under changing or collapsing hash-rate conditions through a responsive but bounded difficulty-control path.

2. Threat Model

CryptEX is designed against a practical threat model rather than an idealized perfect-network model. The most relevant threats at the present stage are the following.

2.1 Hash-rate withdrawal

A dominant miner may disappear abruptly. If difficulty remains calibrated to the previous regime, lower-power miners may be unable to find the next block in reasonable time. The adaptive difficulty controller exists primarily to reduce this failure mode.

2.2 Hash-rate spikes

A temporary surge of mining power may produce a short burst of rapid blocks. If the controller responds too aggressively, the chain may overshoot into difficulty levels that later stall the network when the burst disappears. LWMA damping and EMA smoothing exist to reduce this instability.

2.3 Invalid peers

Peers may repeatedly send invalid headers, invalid blocks, malformed messages, or otherwise disruptive traffic. CryptEX addresses this through peer scoring, temporary bans, score decay, netgroup-aware connection limits, and a bounded synchronization pipeline.

2.4 Competing forks

Multiple candidate branches may be visible simultaneously. CryptEX addresses this by selecting the valid path with maximal cumulative work and by reconstructing and revalidating candidate paths locally before activation.

2.5 Timestamp abuse

Miners may attempt to bias difficulty through pathological timestamps. CryptEX addresses this by clamping solve times, constraining future timestamps, enforcing monotonicity relative to parent blocks, and damping target movement.

2.6 Operator-plane abuse

Remote clients may attempt to abuse the RPC interface through malformed input, oversized requests, or excessive request rate. CryptEX addresses this with authentication, allowlisting, request size limits, and per-IP rate limiting.

The system does not claim to eliminate all attacks. It is designed to make the most likely operational and protocol failure modes explicit, then narrow them through conservative engineering.

3. High-Level Architecture

CryptEX currently consists of the following major subsystems:

1. Consensus and block validation 2. Difficulty control and chain-liveness logic 3. Transaction and UTXO management 4. Wallet and key management 5. Peer-to-peer networking and synchronization 6. Mining and block-template construction 7. Operator interfaces through CLI, config, logging, and JSON-RPC 8. A separate Qt desktop client layered on top of the backend daemon

At runtime, a typical node works as follows:

1. Load block index, headers, and chainstate snapshot from disk. 2. Ensure the baked genesis block exists. 3. Start the P2P listener on TCP port 9333. 4. Optionally start JSON-RPC on TCP port 9332. 5. Discover peers through manual connection, saved peers, DNS seeds, LAN discovery, and peer exchange. 6. Synchronize headers first, then fetch blocks in parallel. 7. Validate candidate chains by recomputing proof of work, Merkle root, transaction validity, reward rules, timestamps, and cumulative work. 8. Activate the strongest valid chain and expose state to CLI, RPC, and GUI consumers.

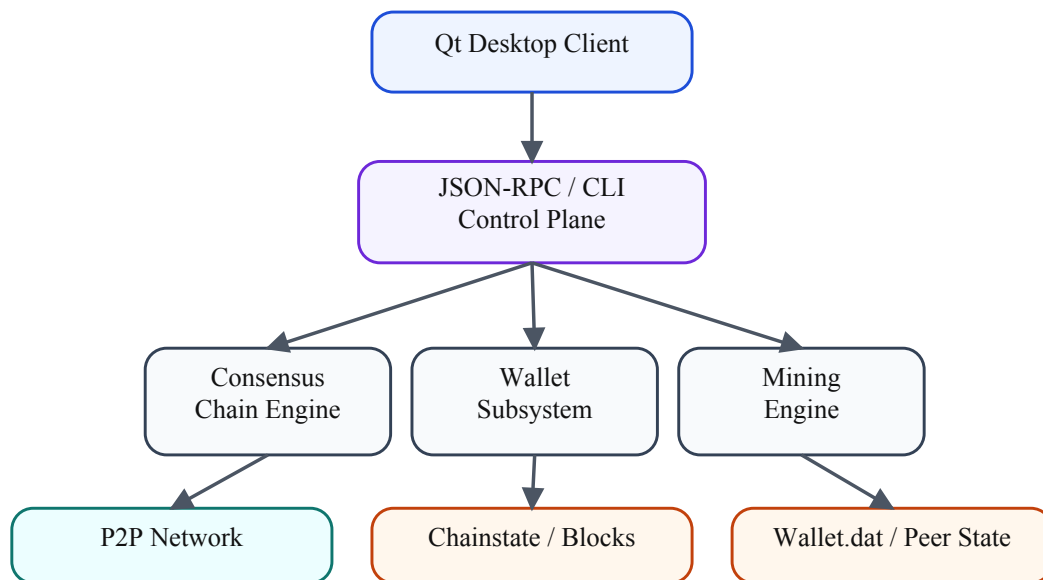


Figure 1. System architecture. The Qt client is separated from the daemon and reaches consensus, wallet, and mining state through the RPC control plane.

4. Detailed Architecture

The earlier sections describe the system at a high level. This section expands that view into the concrete engineering components that make up a running CryptEX node and client stack.

4.1 Consensus and chain engine

The consensus engine is responsible for block validity, chain selection, reward enforcement, and difficulty evaluation. Its primary responsibilities include:

1. genesis handling and baked network parameters
2. block-header parsing and serialization
3. full SHA3-512 proof-of-work comparison
4. compact-target expansion and work accounting
5. cumulative-work chain selection
6. fork-path reconstruction and revalidation
7. coinbase reward ceiling enforcement including fees
8. timestamp, linkage, and ancestry checks

In practice, this means CryptEX does not merely compare heights. It computes candidate chainwork, reconstructs the candidate path back to genesis, and only activates the stronger chain after validating the entire path as a locally verified history.

4.2 Difficulty-control subsystem

The difficulty controller is one of the strongest architectural components in the current codebase. It is designed for networks in which hash rate may not be smooth, continuous, or industrially deep.

The live retarget path combines:

1. a damped LWMA-style estimator
2. an EMA smoothing term
3. a real-time overdue easing term
4. an emergency minimum-difficulty recovery rule

This layered design exists because a young, still-improving, or uneven proof-of-work network must solve two different problems at once:

- respond fast enough when a strong miner exits
- avoid whipsaw or easy manipulation during short bursts of abnormal timing

In other words, this subsystem is not just a tuning detail. It is part of the chain's liveness model.

4.3 Transaction, UTXO, and script subsystem

CryptEX uses a UTXO accounting model. The transaction engine is responsible for:

1. transaction serialization and hashing
2. outpoint resolution
3. value-conservation checks
4. coinbase maturity enforcement
5. signature-hash construction
6. signature verification
7. output creation and UTXO insertion
8. spent-output removal

The script path remains more address-and-signature oriented than a fully generalized Bitcoin Script deployment, but there is already a script-machine implementation in the codebase. Ordinary value transfer is currently validated primarily through ownership mapping and signature correctness rather than unrestricted script programmability.

4.4 Storage and persistence subsystem

CryptEX stores node state as binary files under the system application-data directory. The storage layer includes:

1. active-chain block files by height 2. persisted headers 3. block index metadata 4. chainstate snapshots 5. peer persistence 6. peer reputation and ban persistence 7. encrypted wallet persistence 8. node configuration 9. known-block and side-branch persistence improvements

The key engineering idea here is separation of roles:

- block files preserve active history
- chainstate preserves spend-state reconstruction speed
- index and header files preserve navigation and restart continuity
- peer files preserve reachability and misbehavior memory
- wallet storage remains isolated from chain data while sharing the same datadir

This is what makes the node recoverable after restart without requiring a full rebuild every time.

4.5 Wallet and key-management subsystem

The wallet subsystem is responsible for:

1. encrypted `Wallet.dat` persistence 2. PBKDF2-derived encryption keys 3. AES-256-CBC ciphertext storage 4. BIP39 mnemonic generation and restoration 5. BIP32 hierarchical derivation 6. multi-address accounting 7. private key import and export 8. wallet backup and recovery logic 9. wallet passphrase changes and session management 10. address-pool maintenance for receive/change hygiene

CryptEX therefore treats the wallet as a first-class subsystem rather than a thin address generator bolted onto a node. The presence of encrypted persistence, mnemonic recovery, HD derivation, recovery-oriented save paths, and import/export capabilities gives the wallet layer a much more serious engineering footing than a minimal chain deployment.

4.6 Addressing and representation layer

CryptEX supports multiple address presentations:

1. Base64 2. Base58Check 3. 0x-prefixed hex 4. Bech32

These do not represent four different ownership domains. They are four encodings of the same underlying address payload derived from the compressed secp256k1 public key hash. This distinction is important architecturally:

- the payload defines ownership
- the encoding defines representation

That separation allows the system to remain protocol-consistent while still presenting addresses in forms familiar to different ecosystems and tools.

4.7 Network and synchronization subsystem

The networking layer is implemented with Boost.Asio and includes:

1. TCP peer sessions
2. handshake and version exchange
3. ping / pong liveness checks
4. headers-first synchronization
5. parallel block download
6. peer exchange
7. DNS-seed bootstrap
8. LAN / WLAN peer auto-discovery
9. persisted peer state
10. peer score decay and ban logic
11. netgroup-aware connection diversity
12. network activity controls

This gives CryptEX both a same-LAN usability path and a wider internet bootstrap path. For a practical node implementation, that matters: a network is not only a consensus object, but also an operator experience.

4.8 Mining subsystem

The mining subsystem is responsible for:

1. block-template construction
2. coinbase creation
3. mempool transaction inclusion
4. multi-threaded CPU hashing
5. sync-aware mining startup
6. local and external miner workflows
7. `'GETWORK'` / `'SUBMITWORK'` support
8. `'getblocktemplate'` generation for richer miner integration

The built-in miner is therefore not just a demo loop. It is integrated with tip selection, reward construction, mempool policy, and network-awareness logic. This is also where the adaptive difficulty controller matters operationally: without a good retarget path, the miner becomes a prisoner of stale difficulty after a hash-rate cliff.

4.9 RPC, CLI, and operator control plane

CryptEX exposes a structured operator surface through:

1. CLI commands
2. config-file support
3. structured logging
4. JSON-RPC methods for blockchain, wallet, mining, mempool, and network inspection
5. peer administration actions
6. stop and lifecycle control
7. wallet recovery and maintenance RPCs
8. explorer-style and operator-style introspection methods

This control plane is what turns the codebase from “a chain implementation” into “a node that can actually be operated.” Authentication, rate limits, allowlists, diagnostics, and subsystem-aware logging all belong here.

4.10 Separate GUI client

The current architecture also includes a separate Qt GUI client layered on top of the backend daemon. This separation matters engineering-wise because it divides the system into:

1. backend consensus, networking, mining, and wallet state
2. RPC-driven operator control
3. user-facing wallet and node-management surfaces

That allows the daemon to remain a reusable backend service while the GUI evolves independently as a desktop wallet and node-management surface.

5. Monetary Policy

5.1 Unit of account

CryptEX uses 100,000,000 base units per coin. User-facing wallet balance output is displayed in whole CryptEX units with 8 decimal places, while internal accounting uses satoshi-style integer units for exact arithmetic.

5.2 Target block interval

The target block interval is:

- 600 seconds
- 10 minutes per block

5.3 Block reward

The initial subsidy is:

- 2500.00000000 CryptEX per block

The subsidy halves every:

- 200,000 blocks
- approximately 3.8 years at a 10-minute block interval

Coinbase outputs must mature for:

- 100 blocks

before they become spendable.

5.4 Declared supply and emission curve

The codebase declares a nominal total supply constant of 1,000,000,000 CryptEX. Issuance remains determined by:

1. the initial block reward of 2500 CryptEX 2. the halving interval of 200000 blocks 3. repeated right-shift halving until the reward reaches zero

Let the block height be denoted by h . Let the halving interval be $H = 200000$. Then the number of completed subsidy eras is

$$v(h) = \lfloor h / H \rfloor$$

and the nominal block reward is

$$R(h) = 2500 / 2^{v(h)} \text{ CRX}$$

up to the granularity of the smallest unit.

The resulting total supply is the convergent geometric series

$$500,000,000 + 250,000,000 + 125,000,000 + \square$$

which converges to

$$S_{\infty} = 1,000,000,000 \text{ CRX}$$

6. Consensus Model

6.1 Proof of work

CryptEX uses SHA3-512 for block-header hashing. The consensus-critical proof-of-work comparison uses the full 512-bit digest, not a truncated digest.

The current proof-of-work limit is encoded as:

- ``0x3e00ffff``

This value was chosen to give the network a practical bootstrap difficulty when using a 512-bit hash output.

6.2 Header structure

The serialized block header is 80 bytes and contains:

1. ``version`` (4 bytes)
2. ``prev_block_hash`` (32 bytes)
3. ``merkle_root`` (32 bytes)
4. ``timestamp`` (4 bytes)
5. ``bits`` (4 bytes)
6. ``nonce`` (4 bytes)

This format is intentionally close to Bitcoin's 80-byte header layout.

6.3 Dual hash semantics

CryptEX currently distinguishes between two header-related hashes:

1. ``pow_hash`` - full 512-bit SHA3-512 digest of the serialized header - used for proof-of-work validation - used as the canonical work value and a stored block identity path in major parts of the node
2. ``hash`` - first 32 bytes of the SHA3-512 digest - used as the 256-bit link hash for ``prev_block_hash``

This means CryptEX is already 512-bit where proof of work and work accounting matter most, while block linkage remains 256-bit in the current header schema.

6.4 Chain selection

CryptEX uses most cumulative work, not longest-by-height, to select the active chain. For each block:

1. the compact target is expanded into a full-width target
2. block work is approximated as $(2^{512} - 1) / (\text{target} + 1)$
3. cumulative work is tracked across the known chain graph
4. if a candidate tip has more cumulative work than the current tip, the node reconstructs the full path to

genesis and validates it before activation

6.5 Fork resolution

Fork resolution is conservative:

1. a candidate chain must be fully linked back to genesis 2. every block on the candidate path must validate: - linkage - proof of work - Merkle root - expected difficulty bits - transaction structure - transaction signatures - coinbase reward ceiling including fees - timestamp validity and ancestry consistency 3. only then is the path activated as the canonical chain

This keeps chain switching deterministic and full-node verified.

7. Difficulty Adjustment

CryptEX does not use Bitcoin's original 2016-block retarget rule as its primary live adjustment behavior. Instead, the implementation uses a hybrid per-block retarget path combining a damped LWMA-style estimator, EMA smoothing, real-time overdue easing, and emergency minimum-difficulty recovery.

Current mainnet tuning includes:

- LWMA window: 72 blocks
- intended lookback: about 12 hours
- EMA window: 12 blocks
- minimum solvetime per sample: $\text{target} / 6$
- maximum solvetime per sample: $\text{target} * 6$
- per-step target clamp: between $0.5x$ and $2x$ of the window average target
- emergency minimum-difficulty trigger: $2 * \text{target spacing}$
- final target capped by the protocol proof-of-work limit

Let the nominal target spacing be

$$\tau = 600 \text{ seconds}$$

and let the observed solve time between consecutive blocks be

$$s_i = t_i - t_{i-1}$$

7.1 LWMA component

For a window of length N with linearly increasing weights

$$\omega_i = i, 1 \leq i \leq N$$

the weighted average solve time is

$$s_LWMA = (\sum_{i=1..N} \omega_i s_i) / (\sum_{i=1..N} \omega_i)$$

This term reacts quickly to recent changes, but not all at once.

7.2 EMA component

An exponential moving average is also maintained over recent solve times:

$$E_t = \alpha s_t + (1 - \alpha) E_{t-1}$$

This smooths the response and reduces whipsaw under noisy short-term conditions.

7.3 Real-time overdue easing

If the next candidate block is already overdue relative to the current tip, the system relaxes the target in proportion to the live delay. If

$$\Delta = t_candidate - t_tip$$

then the overdue easing term grows approximately as

$$T_{rt} \approx \Delta / \tau$$

This term matters because it reacts to the actual wall-clock lateness of the next block, not merely to already-mined historical intervals.

7.4 Emergency minimum-difficulty recovery

If lateness becomes sufficiently large, the system falls back all the way to the proof-of-work limit:

$$\Delta \geq 2\tau \implies T_{next} = T_{max}$$

Since $\tau = 600$ seconds on mainnet, the emergency threshold is 1200 seconds.

7.5 Composite controller view

At a high level, the next target can be understood as being chosen from the strongest easing signal that is still bounded by the protocol maximum:

$$T_{next} \approx \min(T_{max}, \max(T_{LWMA}, T_{EMA}, T_{RT}))$$

where 'T_LWMA' is the weighted moving estimate, 'T_EMA' is the smoothed short-horizon estimate, and 'T_RT' is the real-time overdue easing term.

7.6 Why this matters

The point of this controller is not elegance alone. Its practical purpose is this:

- if hash rate remains stable, difficulty should remain stable
- if hash rate changes moderately, difficulty should adapt smoothly
- if hash rate collapses, the chain should still remain mineable
- if the chain is clearly stalled, recovery must be decisive rather than merely hopeful

This is the strongest architectural claim of CryptEX. CryptEX treats liveness under discontinuous hash-rate as a first-class consensus concern, not a secondary tuning detail. The system is built so that a weaker miner can resume block production after a hash-rate cliff instead of waiting indefinitely for a retarget cycle calibrated to an earlier regime.

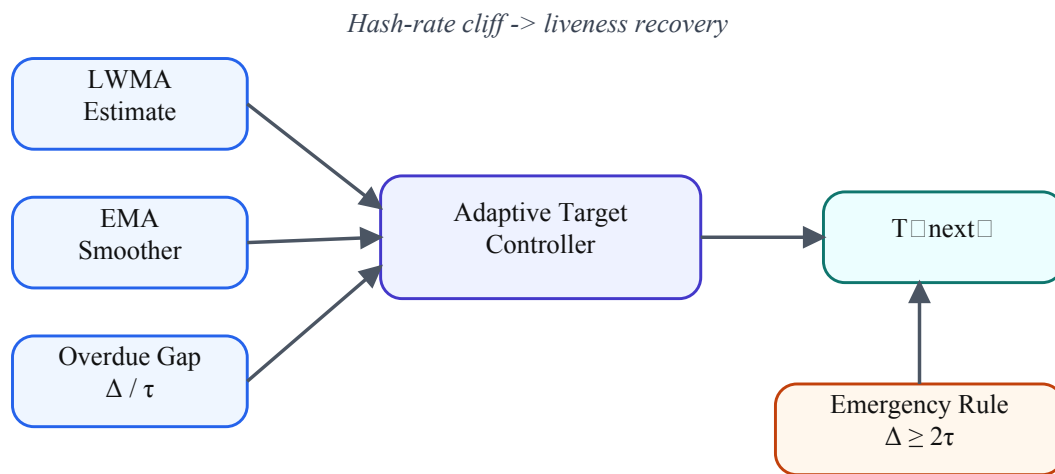


Figure 2. Adaptive liveness controller. Historical weighting, short-horizon smoothing, real-time lateness, and the emergency rule jointly determine the next target.

8. Genesis Block

CryptEX uses a pre-mined genesis block baked into the software.

Current genesis parameters:

- Timestamp: `1741478400`
- Human date: `2025-03-09 00:00:00 UTC`
- Bits: `0x3e00ffff`
- Nonce: `14590424`
- Genesis payout address: `AAECAwQFBgcICQoLDA0ODxAREhM=`

CryptEX also includes a `genesis-mine` utility for offline genesis mining and validation. That tool is used to find and bake in a valid nonce for a selected genesis template.

9. Transaction Model

9.1 UTXO accounting

CryptEX uses a standard unspent transaction output model:

1. transactions consume previously created outputs 2. new outputs are inserted into the UTXO set 3. validation checks signatures, ownership, input existence, value conservation, and maturity rules

Each UTXO entry records:

- output value
- destination address
- creation height
- whether the output came from a coinbase transaction

9.2 Transaction identifiers

Transaction IDs are currently the first 32 bytes of the SHA3-512 digest of the serialized transaction. In other words:

- transaction hashing uses SHA3-512
- txids are still 256-bit identifiers in the current implementation

This is one of the protocol boundaries that still separates full 512-bit proof-of-work identity from the transaction layer.

9.3 Signature and ownership model

CryptEX uses:

- `secp256k1` keys
- ECDSA signatures via OpenSSL

Transaction unlocking today is simpler than a fully general Bitcoin Script execution model. The validation path effectively checks:

1. the referenced UTXO exists 2. the provided public key maps to the destination address payload 3. the signature verifies against the transaction sighash

A more general script engine exists in the codebase, but routine value-transfer validation is currently address-and-signature oriented rather than fully script-programmable.

9.4 Coinbase transactions

A valid block must start with a coinbase transaction. The node enforces:

1. a coinbase must be first 2. the coinbase payout must not exceed block subsidy plus collected fees 3. coinbase outputs require 100 confirmations of maturity before spending

The coinbase ``scriptSig`` includes height, timestamp, and a fragment of the previous hash so coinbase transactions remain unique across blocks.

9.5 OP_RETURN-style payloads

The wallet can optionally attach an ``OP_RETURN:``-prefixed output when creating a transaction. In the current implementation, this output carries a Base64-encoded signed payload artifact derived from the caller-supplied message. It is best understood as a signed attachment mechanism, not a full arbitrary on-chain messaging protocol.

10. Address Format

CryptEX addresses are derived from a 20-byte hash payload:

1. SHA-256 of the compressed secp256k1 public key 2. RIPEMD-160 of that SHA-256 digest 3. encoding of the 20-byte result into one of several supported user-facing formats

The key architectural point is that the payload remains the same while the display encoding changes.

If a compressed public key is denoted by P , then the address payload is derived by

$$A = \text{RIPEMD160}(\text{SHA256}(P))$$

Supported encodings currently include:

- Base64
- Base58Check
- 0x-prefixed hex
- Bech32

These do not represent four different ownership domains. They are four encodings of the same underlying address payload.

Example native CryptEX address shape:

- ``37n70q2VSr4H+0dSpMO+hH8HOGg=``

Important properties:

- the payload defines ownership
- the encoding defines representation
- legacy variants can be normalized to the same underlying address payload

11. Wallet Architecture

11.1 Encrypted wallet storage

`Wallet.dat` is encrypted at rest using:

- PBKDF2-HMAC-SHA256 for key derivation
- 100,000 PBKDF2 iterations
- AES-256-CBC for payload encryption

The wallet file contains:

1. a file version
2. a salt
3. an IV
4. ciphertext length
5. encrypted wallet payload

11.2 HD wallets

New wallets are mnemonic-backed HD wallets using:

- BIP39 mnemonic generation and restoration
- BIP32 deterministic derivation

The current derivation path is:

- `m/0'/0/i`

where `i` is the address index.

Wallets can:

1. create a new mnemonic-backed wallet
2. import from a mnemonic
3. derive additional addresses
4. rescan the chain with a configurable gap limit
5. maintain unused-address pools for cleaner receive/change behavior

11.3 Legacy compatibility and multi-format support

The loader keeps backward compatibility with:

- legacy single-key wallets
- older multi-key payload versions
- legacy internal HD wallets used before the BIP32 migration
- wallets whose display format predates newer address-format handling

This makes CryptEX relatively forgiving during wallet format evolution.

11.4 Multi-address accounting and wallet tools

Wallets support multiple addresses in a single encrypted file. Balance reporting is split into:

1. spendable balance
2. immature balance
3. locked balance
4. total balance

The wallet subsystem also supports:

- `dumpprivkey`
- `importprivkey`

- ``backupwallet``
- ``importmnemonic``
- ``walletpassphrasechange``
- ``getunusedaddress``
- wallet-format selection and persistence

11.5 Wallet recovery and persistence safety

Wallet persistence no longer relies on naive direct overwrite. The current save path writes through a temporary file and rotates a backup copy so that a previous usable wallet image remains available if the primary wallet file becomes unreadable.

12. Peer-to-Peer Network

12.1 Transport and implementation

CryptEX networking is implemented with Boost.Asio over TCP. The default peer-to-peer port is:

- ``9333``

Messages are framed with:

- a 4-byte magic value
- a 1-byte message type
- a 4-byte payload length
- the serialized payload

The current mainnet network magic is:

- ``0x43584558``

12.2 Protocol messages

The node currently supports at least the following message types:

1. ``VERSION`` 2. ``VERACK`` 3. ``PING`` 4. ``PONG`` 5. ``GETHEADERS`` 6. ``HEADERS`` 7. ``GETBLOCK`` 8. ``BLOCK`` 9. ``GETPEERS`` 10. ``PEERS`` 11. ``CHAT`` 12. ``GETWORK`` 13. ``SUBMITWORK`` 14. ``INV`` 15. ``GETTX`` 16. ``TX``

12.3 Peer discovery

CryptEX supports several bootstrap paths:

1. manual ``--connect host:port`` 2. saved peers from previous sessions 3. peer exchange through ``GETPEERS`` / ``PEERS`` 4. DNS seeds 5. LAN / WLAN auto-discovery 6. optional external IP detection and self-advertisement

Serialized peer addresses are compact and include:

- 4 bytes IPv4 address
- 2 bytes port
- 1 byte flags

This allows a node to pass known reachable peers to other nodes in a simple binary format.

12.4 LAN / WLAN discovery

CryptEX includes a same-network discovery path so that users on the same local network do not need to understand direct IP entry before two nodes can find one another. Discovery traffic is broadcast periodically on a UDP port offset from the P2P listener. This is an operator-usability feature, not a consensus rule.

12.5 External address discovery

The node can discover its apparent public IP using an HTTP-based detection service. The current defaults are:

- host: ``ifconfig.me``
- port: ``80``
- path: ``/ip``

This is an operational convenience feature, not a consensus rule. Operators should understand that it introduces trust in the chosen external IP service.

12.6 Peer reputation and bans

CryptEX includes a persisted peer reputation system with:

- misbehavior scoring
- automatic bans after threshold crossing
- score decay over time
- ban persistence on disk
- netgroup-aware peer diversity limits

Peer state is stored in:

- ``peer_state.dat``

Known peers are stored in:

- ``peers.dat``

This gives the network basic resilience against invalid payloads, malformed protocol messages, and spammy peers.

12.7 Public and private chat

CryptEX includes a non-consensus chat transport supporting:

- signed public chat
- signed and encrypted private chat
- inbox and history retrieval
- peer-routed delivery

The chat subsystem is non-consensus and exists as an auxiliary authenticated messaging layer between nodes.

Public chat messages are authenticated with the sender wallet key. Private chat messages are authenticated and encrypted using an ECDH-derived AES-256-GCM session key bound to the recipient public key. Timestamp and nonce fields are included to reduce replay risk.

LAN / DNS / saved peers

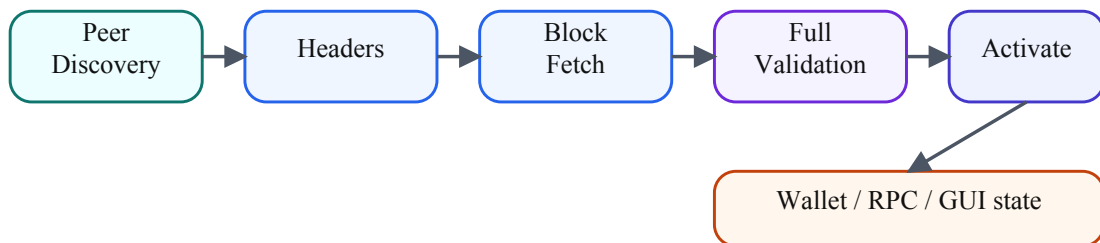


Figure 3. Synchronization pipeline. Discovery, header acquisition, block download, validation, and activation are separated before state is exposed to wallet and GUI surfaces.

13. Synchronization and Block Propagation

13.1 Headers-first sync

CryptEX uses a headers-first synchronization strategy:

1. exchange best heights during version handshake
2. request headers using a block locator
3. validate header continuity
4. queue missing blocks for download
5. download blocks in parallel

Current operational limits include:

- up to 2,000 headers per 'HEADERS' message
- up to 8 parallel block downloads

13.2 Block locators

Block locators are built from the active chain using a stepped-back sequence similar to Bitcoin's locator approach:

1. recent blocks are listed densely
2. older history becomes exponentially sparser

This allows a reconnecting peer to find its last common ancestor efficiently.

13.3 Inventory exchange

Transactions and blocks are announced through inventory messages:

- blocks use the canonical proof-of-work block identity path
- transactions use current 256-bit txids

This reflects the current mixed-width state of the protocol.

13.4 Approval and sync state

The node tracks approval and synchronization state relative to observed peer height, active tip, queued downloads, and known peer set. This state is surfaced into wallet accounting, GUI presentation, and sync-aware mining behavior.

14. Mempool Policy

CryptEX enforces a standardness and fee policy for relay and mempool admission. Current policy includes:

1. no coinbase transactions in mempool
2. minimum relay fee of 1,000 base units per kB
3. maximum standard transaction size of 100,000 bytes
4. input limit of 128
5. output limit of 128
6. maximum scriptSig size of 1,650 bytes
7. dust rejection below 546 base units
8. only one 'OP_RETURN' output per standard transaction
9. mempool conflict detection for double-spends
10. orphan transaction storage and later promotion when parents arrive
11. fee-rate-based eviction when mempool limits are reached

The mempool is therefore policy-aware, not just a raw transaction bucket.

15. Storage Model

15.1 On-disk files

CryptEX stores blockchain state as binary files. The active chain data directory contains files such as:

1. 'blocks/blk<height>.dat'
2. 'headers.dat'
3. 'index.dat'
4. 'chainstate.dat'
5. 'peers.dat'
6. 'peer_state.dat'
7. 'Wallet.dat'
8. 'cryptex.conf'
9. local checkpoint and sync-approval state files

15.2 Block storage

Active-chain blocks are stored by height as:

- 'blk0.dat'
- 'blk1.dat'
- 'blk2.dat'

- and so on

This gives a transparent active-chain storage model. The node rewrites canonical height files when a stronger chain becomes active.

15.3 Side-chain and known-block persistence

CryptEX now includes improved persistence for known blocks beyond the current active tip. Active-chain blocks remain stored by height, while known side-branch and hash-addressed access paths are also retained more deliberately than before. This is still improving, but it is no longer limited to a purely in-memory side-branch view.

15.4 Chainstate snapshot

CryptEX persists a chainstate snapshot to:

- ``chainstate.dat``

This file stores:

1. tip metadata
2. the active UTXO set
3. a checksum

On startup, the node first tries to load ``chainstate.dat``. If the snapshot is missing, stale, or corrupt, it rebuilds the UTXO set from the active block files.

15.5 Default data locations

By default, CryptEX stores data in system application-data locations:

- macOS: ``~/Library/Application Support/CryptEX``
- Linux: ``$XDG_DATA_HOME/CryptEX`` or ``~/.local/share/CryptEX``
- Windows: ``%APPDATA%\\CryptEX``

This avoids accidental chain loss when binaries or build directories are deleted.

16. Mining

16.1 Local CPU mining

CryptEX includes a built-in CPU miner that:

1. builds a block template from the active tip
2. creates a unique coinbase
3. includes mempool transactions until block size is reached
4. hashes candidate headers with SHA3-512
5. compares the full 512-bit digest against the target

Mining supports:

- multi-threaded CPU use
- configurable thread count
- infinite mining mode with ``--cycles 0``

- `--block-cycles`` chained block mining
- debug output for hash-rate and current proof-of-work attempts
- sync-aware startup when peers are present

16.2 External miner interface

CryptEX exposes both:

- a ``GETWORK` / `SUBMITWORK`` flow over the P2P layer
- a ``getblocktemplate`` path over JSON-RPC

This provides both a lightweight external-miner path and a more structured template-based operator path.

Important boundary:

- CryptEX exposes external-miner integration support
- CryptEX is still improving toward more complete pool-grade tooling
- CryptEX currently does not implement a decentralized P2Pool-style mining architecture

17. JSON-RPC and Operator Interface

17.1 RPC transport

CryptEX exposes an HTTP JSON-RPC server with:

- configurable bind address
- configurable port
- Basic authentication
- request size limits
- per-IP rate limiting

Default RPC port:

- ``9332``

For normal operation, RPC should remain bound to ``127.0.0.1`` unless a trusted network perimeter is in place.

17.2 RPC coverage

The current RPC surface includes:

Blockchain and node inspection

- ``getblockcount``
- ``getbestblockhash``

- `getblockhash`
- `getblockheader`
- `getblock`
- `getblockchaininfo`
- `getchaintips`
- `getdifficulty`
- `getpeerinfo`
- `getnetworkinfo`
- `getmininginfo`
- `getmempoolinfo`
- `getportmappinginfo`
- `getpeergraph`
- `getcheckpointinfo`

Transaction and mempool operations

- `getrawmempool`
- `getrawtransaction`
- `gettxout`
- `decoderawtransaction`
- `sendrawtransaction`
- `submitblock`
- `getblocktemplate`

Wallet operations

- `getwalletinfo`
- `getbalance`
- `listunspent`
- `getnewaddress`
- `getunusedaddress`
- `sendtoaddress`
- `dumpprivkey`
- `importprivkey`

- ``importmnemonic``
- ``backupwallet``
- ``walletpassphrasechange``
- ``rescanwallet``
- ``listwallets``
- ``setwalletformat``

Explorer and operator-facing introspection

- ``getrecentblocks``
- ``getaddresssummary``
- ``getaddresstxids``
- ``searchchain``

Chat operations

- public chat send/history
- private chat send/inbox/history
- chat status and routing inspection

Peer administration

- ``addnode``
- ``setban``
- ``clearbanned``
- ``listbanned``
- ``stop``

17.3 Config and logging

CryptEX supports a configuration file:

- ``cryptex.conf``

and structured logging with:

- configurable log level
- console/file output
- JSON log mode
- subsystem filters

This is important for node operations, debugging, and deployments across multiple machines.

18. Security Properties

CryptEX's security posture today includes:

1. full local proof-of-work validation 2. full Merkle-root recomputation on block validation 3. signature verification on transaction spends 4. coinbase maturity enforcement 5. cumulative-work-based chain selection 6. binary wallet encryption at rest 7. wallet backup and recovery paths 8. peer misbehavior scoring and persisted bans 9. rate-limited and authenticated RPC access 10. checksummed chainstate, header, and index persistence 11. timestamp sanity checks in the block-validation path 12. local checkpoint and reorg-safety mechanisms

These are meaningful full-node properties. A node does not simply trust remote peers for chain validity.

19. Current Implementation Boundaries

CryptEX already has a solid base, but the current code also has boundaries that users should understand clearly.

19.1 Full 512-bit proof of work, partial 512-bit data model

CryptEX is fully 512-bit where proof of work and chainwork matter most:

- full 512-bit PoW digest
- 512-bit target/work arithmetic
- 512-bit proof-of-work comparison path

However, the following are still 256-bit in the current implementation:

- ``prev_block_hash`` linkage
- transaction IDs
- Merkle root leaves and internal Merkle combinations
- outpoint transaction references

So the network is best described as a 512-bit proof-of-work chain with a partially 256-bit transaction and header-linkage model.

19.2 Emission tradeoff

The 1 billion coin target is achieved by pairing a 2,500-coin starting reward with a 200,000-block halving interval. This is a deliberate monetary-policy choice, not a bug.

19.3 Side-branch persistence

Active-chain blocks are persisted by height. Known side-branch and hash-addressed block persistence are improving, but they are still evolving compared with the fully mature archival model of a long-running production chain like Bitcoin Core.

19.4 RPC transport security

RPC uses Basic authentication over HTTP. It is suitable for localhost or a trusted private network, but it is not a replacement for TLS, a VPN, or SSH tunneling.

19.5 External IP detection trust

Automatic public-IP detection depends on an external service. This helps usability, but it is a trust and privacy tradeoff.

20. Why CryptEX Is Distinct

CryptEX is distinct from a minimal clone in several ways:

1. SHA3-512 proof of work with full-width target comparison
2. a backend daemon plus separate desktop GUI client
3. a hybrid adaptive difficulty controller designed around liveness under hash-rate cliffs
4. multiple address encodings over a single underlying address payload
5. built-in peer exchange, DNS-seed bootstrap, and LAN / WLAN discovery
6. built-in public/private chat transport over the P2P layer
7. external miner interfaces through both 'GETWORK' and 'getblocktemplate'
8. encrypted HD wallets with mnemonic restore, backup, and recovery paths
9. integrated JSON-RPC and operator tooling

At the same time, it deliberately keeps a Bitcoin-like validation mindset: UTXOs, full-node checks, chainwork, mature coinbase rules, and explicit reorg handling.

21. Conclusion

CryptEX is a serious, still-improving full-node cryptocurrency implementation rather than a minimal chain launcher. Its strongest technical characteristics today are:

1. full 512-bit SHA3 proof of work
2. fork-aware cumulative-work chain selection
3. an adaptive difficulty controller designed for liveness under changing hash-rate conditions
4. headers-first synchronization
5. persisted chainstate and known-block state
6. encrypted HD wallets with mnemonic restore and recovery tooling
7. separate GUI and backend daemon architecture
8. integrated JSON-RPC and operator tooling

Its most important current caveats are also clear:

1. proof-of-work identity is fully 512-bit, but transaction and linkage identities are still partially 256-bit
2. the issuance profile is intentionally much steeper than Bitcoin's historical launch profile
3. side-branch persistence and archival completeness are still improving relative to very mature production chains

Those caveats do not erase the value of the system. They define the next engineering steps. As implemented today, CryptEX is best understood as a compact, security-minded, Bitcoin-inspired SHA3-512 network with adaptive liveness protection, modernized wallet recovery, practical peer discovery, and a clear path toward a more mature production protocol.